Doc. no. P0467R0
Date: 2016-10-13
Project: Programming Language C++
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>
Audience: SG1, Library

Iterator Concerns for Parallel Algorithms

Revision History

Revision 0

Original version of the paper for the 2016 pre-Issaquah mailing.

Table of Contents

1. Introduction

The C++17 CD, N4604, provides additional overloads for most of the algorithms in the `<algorithm>` and `<numeric>` headers, generally by repeating the declaration of the non-parallel form with a leading parallel-policy parameter. It is not clear that sufficient thought was given to the iterator category when providing such additions, and this paper looks into some of the resulting issues. As there are several reasonable directions to proceed from here, formal wording will not be provided until the first revision of this paper, following LEWG review.

2. Problems to be addressed

The main problem is that the parallel algorithms are underspecified. This paper claims that the poor specification naturally leads to those same algorithms being under-constrained, and suggests that different constraints may be appropriate for the serial and parallel forms of the same algorithm.

2.1 The Standard Algorithms Are Underspecified

To start, we must realize that many of the existing standard algorithms are under-constrained, at least when we want to extend their specification to cover the otherwise-unspecified parallel case.

Good examples of algorithms that have had their specification honed over several standard revisions are `adjecent_difference`

and `partial_sum` in the `<numeric>` header. However, most algorithms are more like `copy`, and have poorly stated requirements that must be implicitly reverse-engineered from their *Effects:* clause.

2.2 Input Iterators Invalidate Too Frequently

The basic problem with an Input Iterator is that it does not provide the multi-pass guarantee. This makes it impossible for a parallel algorithm to work with different subsequences at the same time, as it is not possible to advance the iterator to refer to such subsequences without invalidating the iterators used by other threads.

2.3 Copying an Input Range Requires Memory, and Copy Semantics

One workaround for the lack of a multi-pass guarantee is to make copies of subsets of the input range, and dispatch work on those subsets. If this is the intended semantic, which is the understanding of the author, then it would be helpful to call this out in the parallel algorithms wording at the front of clause 25, so that future discussion in the Library Working Group dealing with parallel algorithms is informed of a deliberate design decision, calling out the runtime trade-offs and overheads accepted for the parallel forms. However, permission to create copies is not sufficient, as the algorithms do not currently require those elements to be `CopyConstructible`. For example, it is expected that the `move` algorithm work with `move_iterator`s, possibly over sequences of `unique_ptr`s.

2.4 Writing to an Output Iterator Implies a Serialization Stage

Algorithms that write to Output Iterators do not currently require `CopyConstrutible` elements; for example, an `ostream_iterator` requires just a reference in order to write the result to a stream.

Consider the following example:

```
template <typename T>
void WriteVector(std::vector<T> const & v, std::ostream & os)  {
   copy(v.begin(), v.end(), std::ostream_iterator(os, ", "));
}
```

Despite the name of the algorithm being `copy`, we do not expect to make any actual copies executing with these iterators. For a clearer example, consider the following:

```
template <typename T>
void WriteVector(std::vector<std::unique_ptr<T>> const & v, std::ostream & os)  {
   std::transform(v.begin(), v.end(),
                  std::ostream_iterator(os, ", "),
                  [](auto const &ptr){ return ptr.get(); });
}
```

In this case, we cannot copy elements from the source range, as `unique_ptr` does not have a usable copy constructor, but there is still an expectation that the algorithm should compile and run.

An output sequence supplied through a simple output iterator, such as the aforementioned `ostream_iterator`, does not offer the multi-pass guarantee; so again, output must be collected and output serially. It would be very surprising for the `copy` algorithms to write a permuted sequence, or for `merge` to produce a non-sorted output.

3. Proposed Resolution: Promote All Iterators to At Least Forward Iterators

The simplest short-term fix is to require that no iterator for a parallel algorithm has a category less capable than a Forward Iterator. This change guarantees that all iterators in parallel algorithms return a true reference, i.e., no copies are ever required, and the multi-pass guarantee allows the implementation to `advance` the destination (output) iterators without requiring additional storage to queue results for serialization.

This solution is recommended as it preserves the ability to weaken the constraints on iterators in a future library, once the implementation requirements are clear. It also grants time to the Library Working Group to properly document the constraints on the existing non-parallel algorithms.

3.1 Which Algorithms Are Affected?

Roughly 40 parallel algorithms use either input or output iterators in their interface, most of which use both.

Parallel Algorithms with Input Iterators and an Implicit Specification

The following algorithms all consume input ranges, may write to a simple output iterator, and make no clear requirement that the element type be copyable. Their specification is entirely implicit, not appearing beyond their corresponding header synopsis.

```
all_of
any_of
none_of
find
find_if
find_if_not
find_first_of
count
count_if
mismatch
equal
copy_n
copy_if
transform
replace_copy
replace_copy_if
remove_copy
remove_copy_if
partial_sort_copy
merge
includes
set_union
set_intersection
set_difference
set_symmetric_difference
lexicographical_compare
reduce
transform_reduce
inner_product
exclusive_scan
inclusive_scan
transform_exclusive_scan
transform_inclusive_scan
```

Parallel Algorithms with Input Iterators and an Explicit Specification

The following algorithms all consume input ranges, may write to a simple output iterator, and do not make a clear requirement that the element type be copyable. They do provide a distinct specification for the parallel form having a subtly different interface, such as`for_each`, or slightly different requirements, such as `copy`.

```
for_each
for_each_n
copy
move
```

Parallel Algorithms Writing Through Output Iterators

The following algorithms not already mentioned as consuming an input sequence require a serial (potentially copying) stage to write through an output iterator.

```
fill_n
generate_n
reverse_copy
rotate_copy
```

Potentially Well-specified Algorithms

The following algorithms over input ranges have implicitly specified parallel overloads, but may have a strong enough serial specification that the requirements are sufficient to allow for some parallel execution. However, even in these cases, careful review will likely suggest a minimal tweaking for clear support.

```
unique_copy
partition_copy
partial_sum
adjacent_difference
```

4. Alternatives Solutions

Several other resolutions were considered preparing this paper, and should be considered alongside the recommended resolution above.

4.1 Do Nothing

There is an intent that it should be very simple to turn serial C++ code into parallel code by simply adding a parallel-policy argument to the call of an algorithm. This becomes more troublesome if the user has to consider the potential for different requirements on the serial and parallel forms.

In particular, the standard gives no mandate that even those algorithms called with iterators that reveal embarrassing potential for parallelism should actually distribute the work, not least because the standard does not exclude environments that support only a single thread of execution. From this perspective, the parallel execution policies are no more than a hint to the library, like the `inline` keyword is a hint to the compiler. They allow for the relaxation of certain constraints (such as the ODR in the case of `inline`), subject to further constraints (e.g., identical definitions) that will retain proper behavior.

In this case, the promise made, beyond being a hint to the library that concurrent execution is desired, is the guarantee that passed function objects and iterators do not introduce data races when executed on different (unsynchronized) threads. This guarantee is a constraint on the user of the algorithm, rather than on the library implementation.

The author rejects this direction, as making a false offer of parallelism where serial implementations are effectively mandated is even more misleading. It would be better for the caller to confront the issues with their data structures that inhibit effective parallelism. Also, to some extent this risk is already assumed in the several algorithms that already have a different interface or contract for the serial and parallel forms, such as `for_each` and `copy`.

A further concern with this approach is that it has not been applied consistently through the library, as there are several algorithms with no parallel overloads, which could easily be given parallel overloads on the assumption that all known implementations would be sequential, but open to QoI enhancement in the future.

4.1a Add the Missing Parallel Overloads

This is an extension of the *do nothing* solution, which addresses the concern that the abstraction of parallel policy as a hint does not work as long as there are some algorithms lacking the parallel policy overloads. Therefore, the policy parameter would be added to the remaining algorithms called out as *not* supported in the Parallelism Extensions TS.

4.2 Revise Requirements on all Algorithms

This resolution would likely require more effort from the Library Working Group than is available within the ballot resolution period for C++17. It would involve first correctly specifying all of the existing serial algorithms with a level of detail that is missing today. Once those specifications are complete, the parallel form of those same algorithms can be reviewed, and specified with additional constraints where necessary, such as the ability to take copies in order to create sub-tasks.

This resolution is the preferred long-term goal of the paper author, but seems beyond the scope of what can be achieved in two standard meetings, while addressing all of the other outstanding ballot comments. One possible result of this approach might be weaker requirements and a mandate to avoid undue serialization in the event of algorithms being called with Forward Iterators (or better) rather than simple Input or Output Iterators. At the least, it is an opportunity for the standard to acknowledge the greater capabilities that would come from such iterators.

4.3 Blanket Wording Permitting Copies in Parallel Algorithms

A half-way house would be to add additional blanket wording to the parallel algorithms front-matter, that says that any algorithm with a parallel policy overload taking input or output iterators as parameters, requires that the element type for such iterators be `CopyConstructible`.

While this direction would the progress from the status-quo, it suffers from still being an awkward specification for readers of the standard, who must be aware that there are now requirements on algorithms documented in several other places than the algorithm itself is specified, especially for parallel forms that are not otherwise specified at all. This could be partially alleviated by applying a blanket edit to actually specify those algorithms below the serial form, adding the `CopyConstructible` constraints in each case. Such a solution would still suffer from interactions with the under-specification of the serial forms though.

A trickier problem is that Output Iterators do not actually have an element type, so it is not clear what `CopyConstructible` constraint should apply to.

5. Implementation Experience

P0024r1 provide links to existing practice prior to standardization. How do these implementations handle the problems highlighted in this paper?

5.1 Force Serial Execution

The Microsoft implementation at CodePlex simply forces all calls with input iterators into the sequential policy overloads - I did not find special handling for output iterators, but could easily have missed it in the time I had for this review.

5.2 Copy Elements into Subranges

The Khronos Sycl Parallel STL appears to try to copy elements into subranges, and performs parallel execution with the subranges. The additional requirements for copyable elements do not appear to be documented, and as far as I can tell, calling with an incompatible element type would lead to a failure instantiating the template, deep in the implementation details. Libraries using this approach are best place to provide insights into the missing requirements, if that is the preferred direction.

5.3 Ignore the Issue (bug?)

Other libraries seemed to simply ignore the issue, working with potentially invalidated input (or output) iterators and risking undefined behavior. i.e., they have bugs, and lend no insight for a likely solution.

I am not calling out which of the unreferenced libraries fall into the categories above, as I did not have time for a detailed review, but these three categories appear to cover the range of the behavior I encountered.

6. Formal Wording

Wording will follow once the LEWG has given guidance on the preferred direction.

7. Acknowledgements

Thanks to David Sankel and Dietmar Kühl for early reviews of this paper, without necessarily endorsing it.

8. References

- N4606 (Proposed) Working Draft for C++
- N4507 C++ Extensions for Parallelism TS
- P0024r1 The Parallelism TS Should be Standardized, Jared Hoberock
- http://parallelstl.codeplex.com Parallel STL at CodePlex
- https://github.com/KhronosGroup/SyclParallelSTL Khronos Group SyclParallelStl